

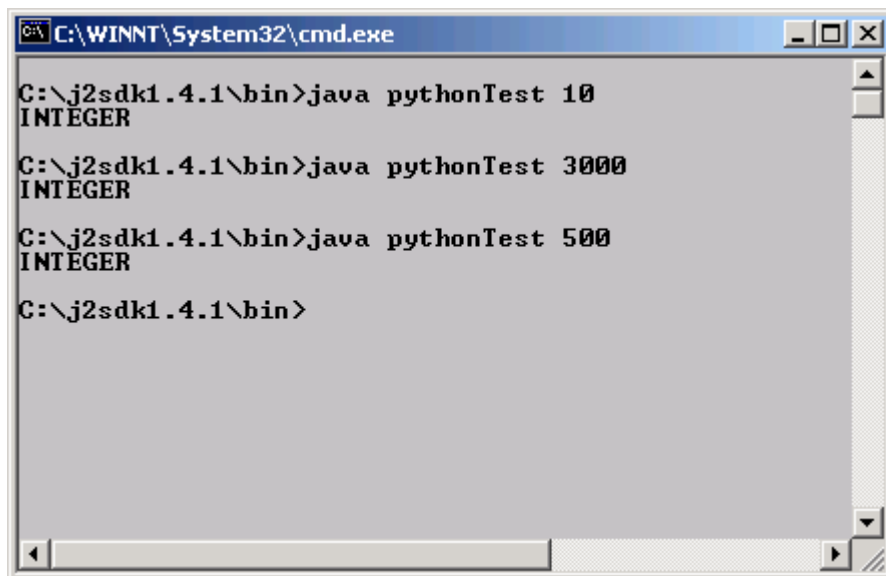
บทที่ 5

ผลการทดสอบ

ในการพัฒนาโปรแกรม หรือซอฟต์แวร์นั้น มีขั้นตอนในการทำงานอยู่หลายขั้นตอน ซึ่งการทดสอบความถูกต้องของโปรแกรมนั้นก็เป็นหนึ่งในหลายขั้นตอนของการพัฒนาโปรแกรม ซึ่งนับได้ว่ามีความสำคัญมากเพราะจะทำให้เรามั่นใจได้ว่าโปรแกรมที่เราทำการพัฒนาขึ้นมานั้นสามารถนำไปใช้งานได้ถูกต้อง อีกทั้งขั้นตอนการทดสอบความถูกต้องของโปรแกรมนั้นจะทำให้เราสามารถที่จะค้นพบข้อผิดพลาดบางอย่าง ซึ่งทำให้เราสามารถแก้ไขได้ก่อนที่จะมีการนำไปใช้งานได้จริง

5.1 การทดสอบความถูกต้องของกลุ่มคำที่ได้ทำการกำหนดโดยใช้เรกกูลาร์เอ็กซ์เพรสชัน

การทดสอบความถูกต้องของกลุ่มคำนั้นเราจะกระทำโดยการนำเฉพาะโค้ดในส่วนของการตรวจจับกลุ่มคำมาทดลองใส่กลุ่มคำทั้งแบบที่ถูก และแบบที่ผิดหลายๆ ตัวอย่าง และทำการแสดงชื่อของเรกกูลาร์เอ็กซ์เพรสชันนั้นๆ ออกมาทางเอาต์พุตถ้าคำที่ทำการทดลองตรงตามเรกกูลาร์เอ็กซ์เพรสชันตัวนั้นๆ และแสดงคำว่า “ERROR” ออกมาทางเอาต์พุตหากคำที่ทดลองไม่ถูกต้องตามเรกกูลาร์เอ็กซ์เพรสชันใดๆ เลย ซึ่งจากการทดลองกับกลุ่มคำต่างๆ จึงได้ผลออกมาดังต่อไปนี้



```
C:\WINNT\System32\cmd.exe

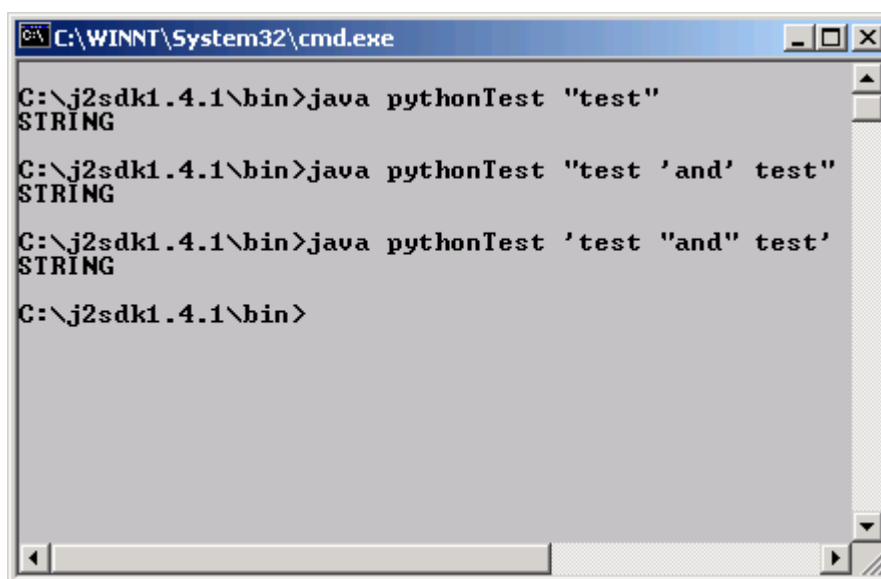
C:\j2sdk1.4.1\bin>java pythonTest 10
INTEGER

C:\j2sdk1.4.1\bin>java pythonTest 3000
INTEGER

C:\j2sdk1.4.1\bin>java pythonTest 500
INTEGER

C:\j2sdk1.4.1\bin>
```

รูปที่ 5-1 แสดงตัวอย่างการทดสอบเรกกูลาร์เอ็กซ์เพรสชันของข้อมูลชนิด integer



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest "test"
STRING

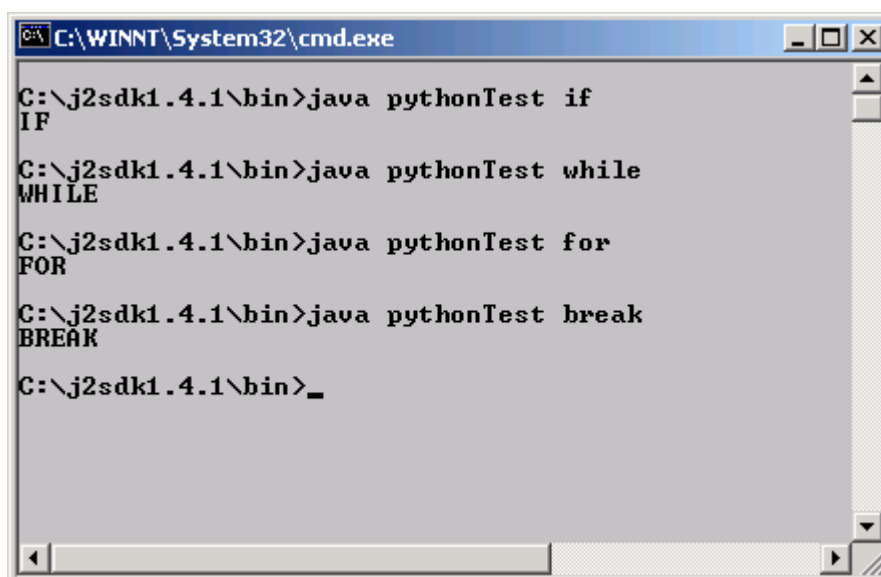
C:\j2sdk1.4.1\bin>java pythonTest "test 'and' test"
STRING

C:\j2sdk1.4.1\bin>java pythonTest 'test "and" test'
STRING

C:\j2sdk1.4.1\bin>

```

รูปที่ 5-2 แสดงตัวอย่างการทดสอบเรกูลาร์เอ็กซ์เพรสชันของข้อมูลชนิด *string*



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest if
IF

C:\j2sdk1.4.1\bin>java pythonTest while
WHILE

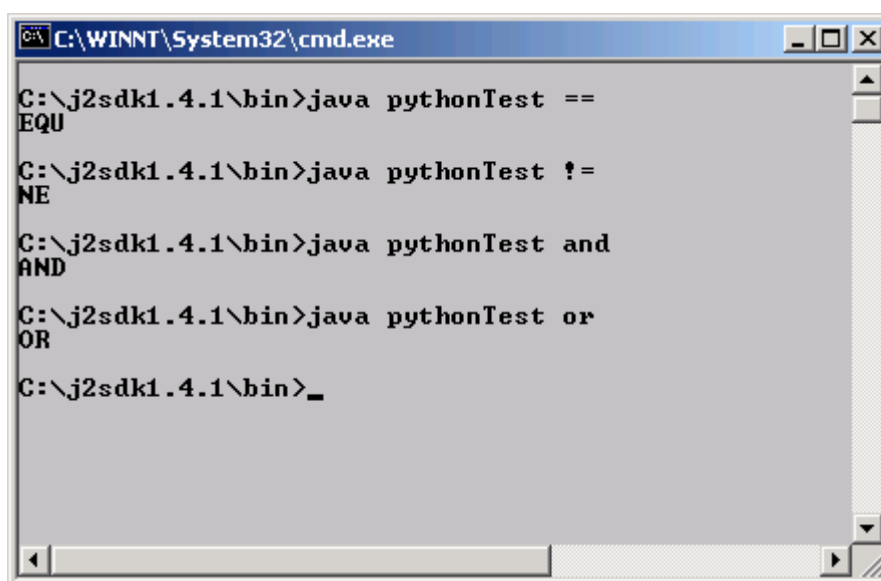
C:\j2sdk1.4.1\bin>java pythonTest for
FOR

C:\j2sdk1.4.1\bin>java pythonTest break
BREAK

C:\j2sdk1.4.1\bin>_

```

รูปที่ 5-3 แสดงตัวอย่างการทดสอบเรกูลาร์เอ็กซ์เพรสชันของคำสงวนต่างๆ



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest ==
EQU

C:\j2sdk1.4.1\bin>java pythonTest !=
NE

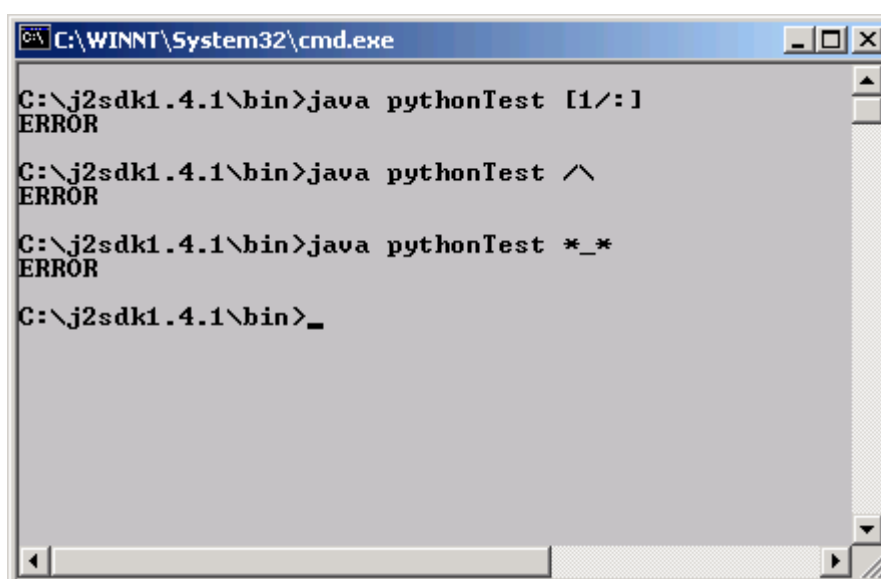
C:\j2sdk1.4.1\bin>java pythonTest and
AND

C:\j2sdk1.4.1\bin>java pythonTest or
OR

C:\j2sdk1.4.1\bin>_

```

รูปที่ 5-4 แสดงตัวอย่างการทดสอบเรกกูลาร์เอ็กซ์เพรสชันของตัวกระทำแบบต่างๆ



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest [1/:]
ERROR

C:\j2sdk1.4.1\bin>java pythonTest /\
ERROR

C:\j2sdk1.4.1\bin>java pythonTest *_*
ERROR

C:\j2sdk1.4.1\bin>_

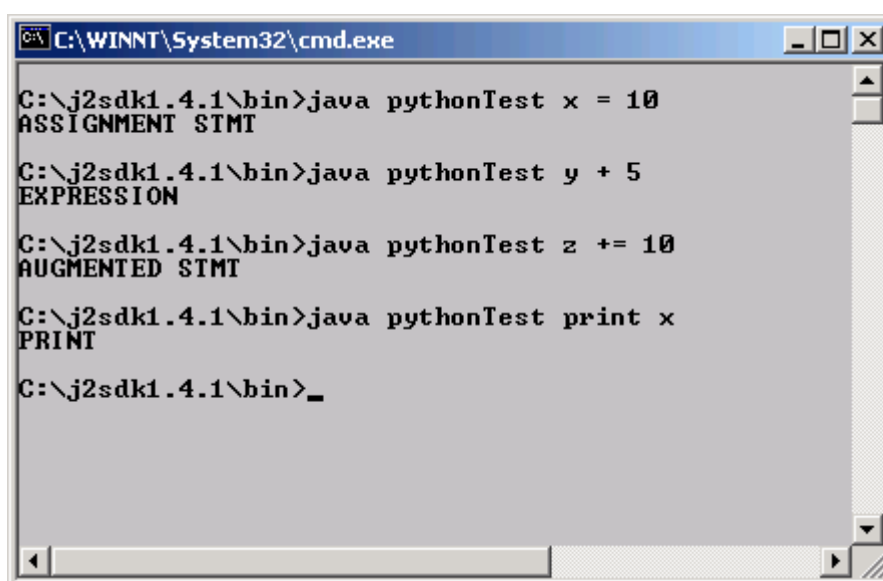
```

รูปที่ 5-5 แสดงตัวอย่างการทดสอบคำที่ไม่ถูกต้องตามเรกกูลาร์เอ็กซ์เพรสชันใดเลย

5.2 การทดสอบความถูกต้องของประโยคที่ได้ทำการกำหนดโดยใช้บีเอ็นเอฟ

การทดสอบความถูกต้องของประโยคนั้นเราจะกระทำโดยการนำส่วนของการตรวจจับกลุ่มคำมา รวมกับส่วนของการตรวจสอบประโยคที่ทำการสร้างขึ้น โดยจะมีการทดลองใส่ประโยคทั้งแบบที่ถูกต้อง และ แบบที่ผิดหลายๆ ตัวอย่าง และทำการแสดงชื่อของบีเอ็นเอฟนั้นๆ ออกมาทางเอาต์พุตถ้าประโยคที่ทำการ ทดลองตรงตามบีเอ็นเอฟตัวนั้นๆ และแสดงคำว่า “ERROR” ออกมาทางเอาต์พุตหากประโยคที่ทดลองไม่ ถูกต้องตามบีเอ็นเอฟใดๆ เลย

ซึ่งจากการทดลองกับประโยคต่างๆ จึงได้ผลออกมาดังต่อไปนี้



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest x = 10
ASSIGNMENT STMT

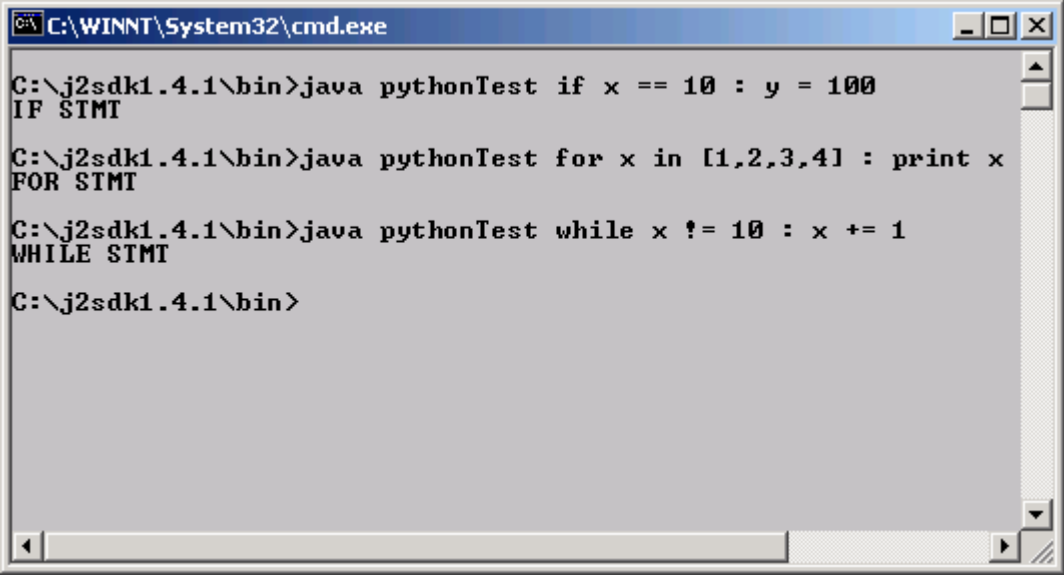
C:\j2sdk1.4.1\bin>java pythonTest y + 5
EXPRESSION

C:\j2sdk1.4.1\bin>java pythonTest z += 10
AUGMENTED STMT

C:\j2sdk1.4.1\bin>java pythonTest print x
PRINT

C:\j2sdk1.4.1\bin>_
  
```

รูปที่ 5-6 แสดงตัวอย่างการทดสอบประโยคที่เป็นคำสั่งแบบตามลำดับต่างๆ ไป



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest if x == 10 : y = 100
IF STMT

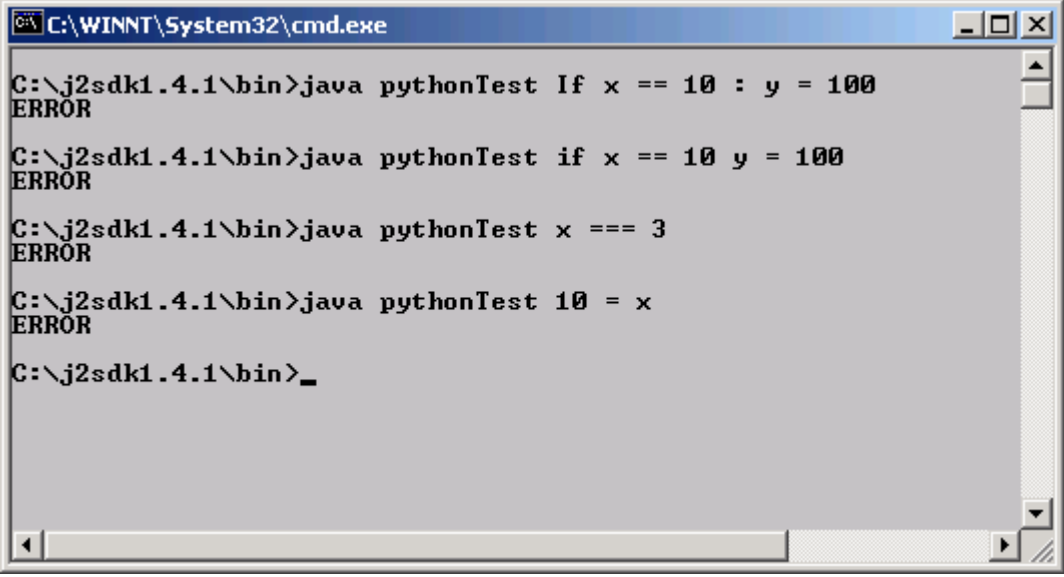
C:\j2sdk1.4.1\bin>java pythonTest for x in [1,2,3,4] : print x
FOR STMT

C:\j2sdk1.4.1\bin>java pythonTest while x != 10 : x += 1
WHILE STMT

C:\j2sdk1.4.1\bin>

```

รูปที่ 5-7 แสดงตัวอย่างการทดสอบประโยคที่เป็นคำสั่งแบบเงื่อนไขและแบบวนรอบ



```

C:\WINNT\System32\cmd.exe

C:\j2sdk1.4.1\bin>java pythonTest If x == 10 : y = 100
ERROR

C:\j2sdk1.4.1\bin>java pythonTest if x == 10 y = 100
ERROR

C:\j2sdk1.4.1\bin>java pythonTest x === 3
ERROR

C:\j2sdk1.4.1\bin>java pythonTest 10 = x
ERROR

C:\j2sdk1.4.1\bin>_

```

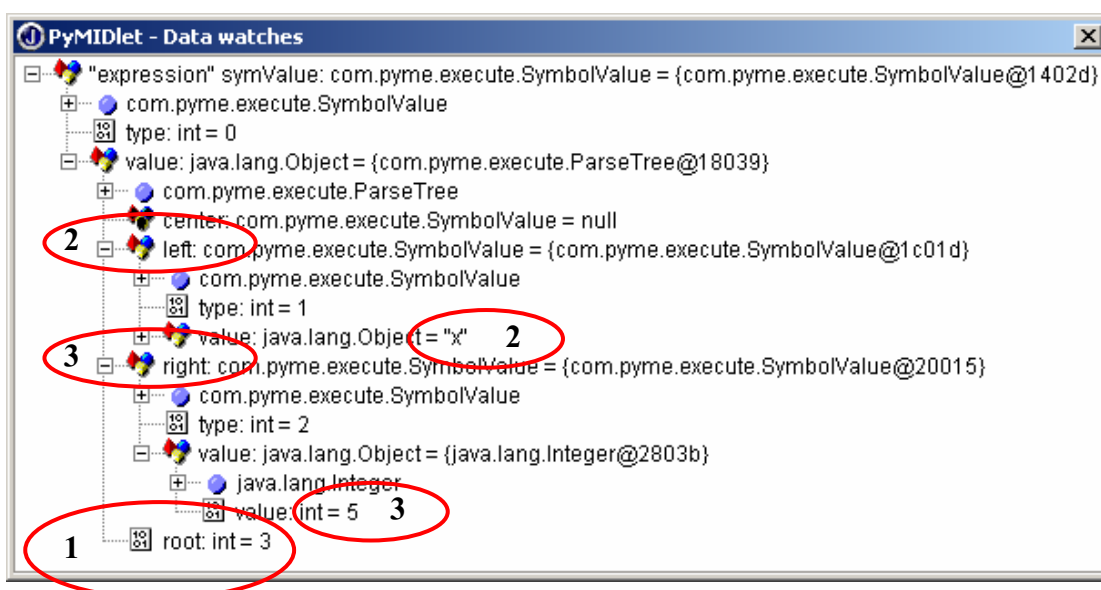
รูปที่ 5-8 แสดงตัวอย่างการทดสอบประโยคที่ไม่ถูกต้องตามบีเอ็นเอฟ

5.3 การทดสอบความถูกต้องของพาร์ซทรี

การทดสอบความถูกต้องของพาร์ซทรีนั้นเราจะกระทำโดยการเพิ่มการสร้างพาร์ซทรีเข้าไปในส่วนของการตรวจสอบประโยค ซึ่งเราจะทำการดีบั๊ก (debug) ค่าตัวแปรที่ใช้เป็นพาร์ซทรี โดยหากประโยคมีความถูกต้องโปรแกรมจะทำการสร้างพาร์ซทรีแล้วส่งออกมาการกระทำตามคำสั่งอีกที

ซึ่งจากการทดลองกับประโยคต่างๆ จึงได้ผลออกมาดังต่อไปนี้

5.3.1 ทดลองป้อนคำสั่งเป็น $x + 5$ ได้ผลเป็นดังนี้

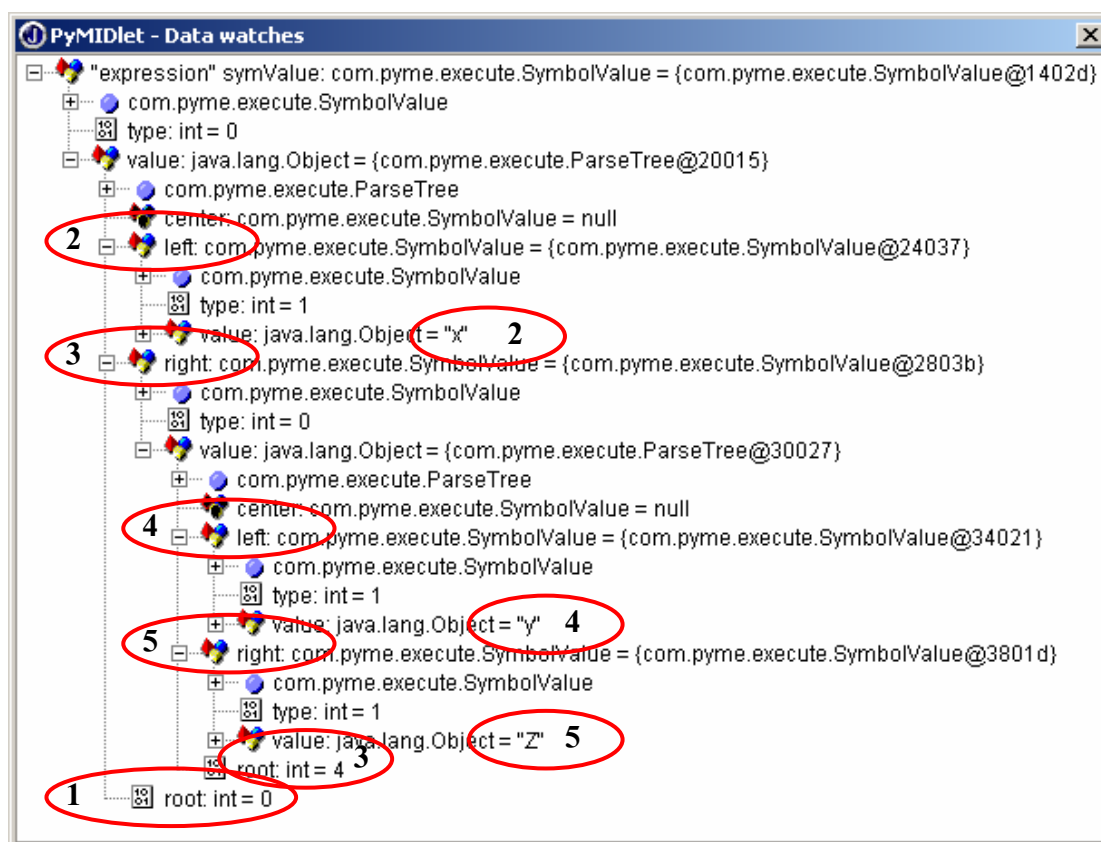


รูปที่ 5-9 แสดงตัวอย่างการทดสอบพาร์ซทรีของประโยค $x + 5$

จากรูปที่ 5-9 นี้มีความหมายดังต่อไปนี้

- หมายเลข 1 แสดงถึงรากของพาร์ซทรีซึ่งมีเท่ากับ 3 ซึ่งได้กำหนดไว้ว่าเป็นพาร์ซทรีของการบวกเลข
- หมายเลข 2 แสดงถึงลูกทางด้านซ้ายของพาร์ซทรีหมายเลข 1 ซึ่งมีค่าเป็น "x"
- หมายเลข 3 แสดงถึงลูกทางด้านขวาของพาร์ซทรีหมายเลข 1 ซึ่งมีค่าเป็น 5

5.3.2 ทดลองป้อนคำสั่งเป็น $x = y - Z$ ได้ผลเป็นดังนี้

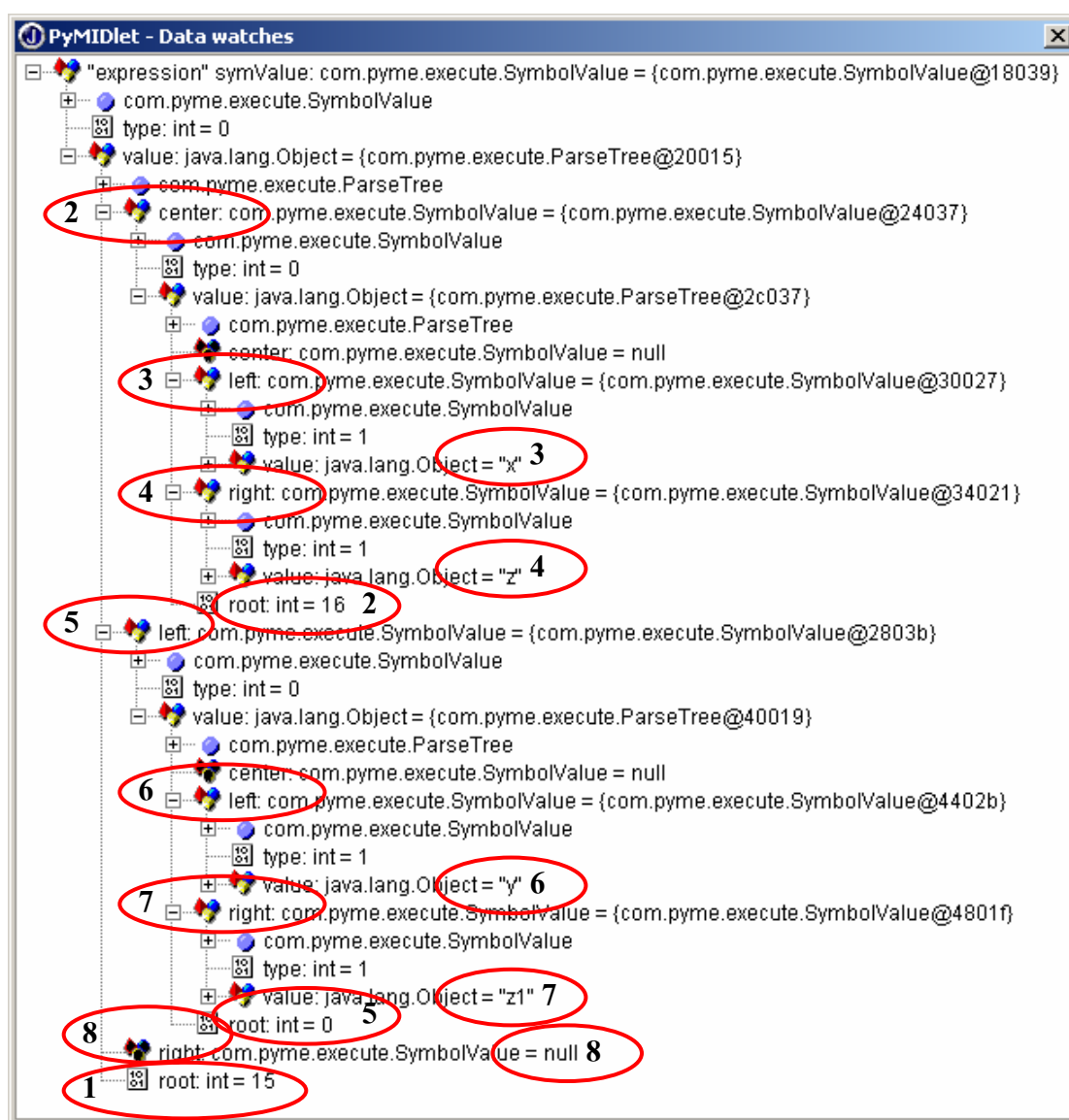


รูปที่ 5-10 แสดงตัวอย่างการทดสอบพาร์ซทรีของประโยค $x = y - Z$

จากรูปที่ 5-10 นี้มีความหมายดังต่อไปนี้

- | | |
|-----------|--|
| หมายเลข 1 | แสดงถึงรากของพาร์ซทรีซึ่งมีเท่ากับ 0 ซึ่งได้กำหนดไว้ว่าเป็นพาร์ซทรีของคำสั่งการกำหนดค่า (assignment statement) |
| หมายเลข 2 | แสดงถึงลูกทางด้านซ้ายของพาร์ซทรีหมายเลข 1 ซึ่งมีค่าเป็น "x" |
| หมายเลข 3 | แสดงถึงลูกทางด้านขวาของพาร์ซทรีหมายเลข 1 ซึ่งมีลักษณะเป็นพาร์ซทรีซึ่งมีรากของพาร์ซทรีมีค่าเป็น 4 ซึ่งได้กำหนดไว้ว่าเป็นพาร์ซทรีของการลบเลข |
| หมายเลข 4 | แสดงถึงลูกทางด้านซ้ายของพาร์ซทรีหมายเลข 3 ซึ่งมีค่าเป็น "y" |
| หมายเลข 5 | แสดงถึงลูกทางด้านขวาของพาร์ซทรีหมายเลข 3 ซึ่งมีค่าเป็น "Z" |

5.3.3 ทดลองป้อนคำสั่งเป็น `if x == z : y = z1` ได้ผลเป็นดังนี้



รูปที่ 5-11 แสดงตัวอย่างการทดสอบพาร์ซทรีของประโยค `if x == z : y = z1`

จากรูปที่ 5-11 นี้มีความหมายดังต่อไปนี้

- หมายเลข 1 แสดงถึงรากของพาร์ซทรีซึ่งมีเท่ากับ 15 ซึ่งได้กำหนดไว้ว่าเป็นพาร์ซทรีของคำสั่ง `if statement`
- หมายเลข 2 แสดงถึงลูกตรงกลางของพาร์ซทรีหมายเลข 1 ซึ่งมีลักษณะเป็นพาร์ซทรี โดยจะเป็นเงื่อนไขของคำสั่ง `if` มีรากของพาร์ซทรีมีค่าเป็น 16 ซึ่งได้กำหนดไว้ว่าเป็นพาร์ซทรีของเงื่อนไขเท่ากับ
- หมายเลข 3 แสดงถึงลูกทางด้านซ้ายของพาร์ซทรีหมายเลข 2 ซึ่งมีค่าเป็น "x"
- หมายเลข 4 แสดงถึงลูกทางด้านขวาของพาร์ซทรีหมายเลข 2 ซึ่งมีค่าเป็น "z"

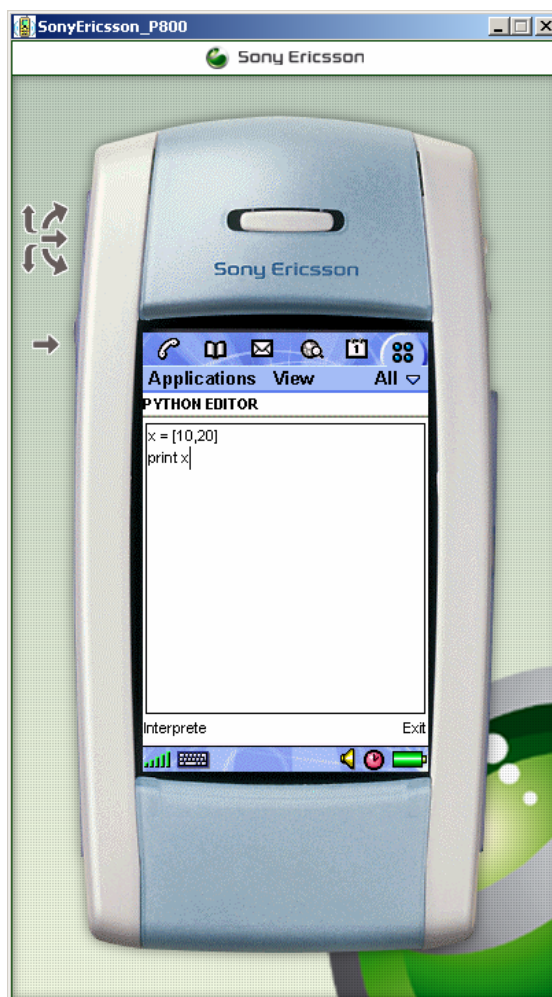
- หมายเลข 5 แสดงถึงลูกทางด้านซ้ายของพาร์เซทรีหมายเลข 1 ซึ่งมีลักษณะเป็นพาร์เซทรีซึ่งมีรากของพาร์เซทรีมีค่าเป็น 0 ซึ่งได้กำหนดไว้ว่าเป็นพาร์เซทรีของของคำสั่งการกำหนดค่า (assignment statement)
- หมายเลข 6 แสดงถึงลูกทางด้านซ้ายของพาร์เซทรีหมายเลข 5 ซึ่งมีค่าเป็น “y”
- หมายเลข 7 แสดงถึงลูกทางด้านขวาของพาร์เซทรีหมายเลข 5 ซึ่งมีค่าเป็น “z1”
- หมายเลข 8 แสดงถึงลูกทางด้านขวาของพาร์เซทรีหมายเลข 1 ซึ่งมีค่าเป็น null ซึ่งนั่นหมายความว่าคำสั่ง if นี้ไม่มี else นั้นเอง

5.4 การทดสอบความถูกต้องของการกระทำตามคำสั่ง และนำไปใช้งานบนมือถือ

การทดสอบความถูกต้องของการกระทำตามคำสั่งพาร์เซทรีนั้นเราจะนำเอาพาร์เซทรีที่ถูกต้องทั้งหมดแล้ว มาทำการประมวลผลและแสดงผลลัพธ์ออกมา โดยเพื่อให้การทดสอบตรงกับความเป็นจริงและพร้อมที่จะนำไปใช้ได้เลย เราจะทำการทดสอบกับโปรแกรมจำลองมือถือ (mobile emulator) และนำไปทดสอบบนมือถือจริงด้วย

ซึ่งจากการทดลองกับคำสั่งต่างๆ จึงได้ผลออกมาดังต่อไปนี้

รูปที่ 5-12 แสดงตัวอย่างการป้อนสคริปต์การกำหนดค่าแล้วแสดงออกทางหน้าจอ





รูปที่ 5-13 แสดงผลลัพธ์จากการอินเทอร์พรีตสคริปต์การกำหนดค่าแล้วแสดงออกทางหน้าจอ



รูปที่ 5-14 แสดงตัวอย่างการป้อนและผลลัพธ์จากสคริปต์วนลูปและแสดงค่า

จากรูปที่ 5-14 เป็นการกำหนดค่าให้กับ x เป็น 10 แล้วทำการวนรอบการทำงานโดยมีเงื่อนไขว่า ถ้า x น้อยกว่า 20 ให้ทำการแสดงค่า x และเพิ่มค่า x ขึ้นอีก 1 ซึ่งก็จะได้ผลลัพธ์เป็นค่า 10 ถึง 19 ตามลำดับ แสดงออกมาทางหน้าจอ



รูปที่ 5-15 แสดงตัวอย่างการป้อนสคริปต์ที่มีความผิดพลาด

จากรูปที่ 5-15 เป็นการสั่งให้โปรแกรมแสดงค่าของ x ออกมาทางหน้าจอ แต่เนื่องจาก x ยังไม่เคยมีการกำหนดค่ามาก่อน ดังนั้นโปรแกรมจึงแสดงข้อความที่บ่งบอกถึงความผิดพลาดว่า ไม่มีตัวแปรที่ชื่อว่า 'x'